

Striver SDE Sheet - Linked List Part II

Quick DSA Revision Notes

Revision format: Approach -> core idea -> time and space complexity. Focus on the optimal pattern before interviews.

1. Intersection of Two Linked Lists

Approach	Core idea	Time	Space
Brute force	Compare every node of the first list with every node of the second list. Compare node addresses, not values.	$O(N*M)$	$O(1)$
Hashing	Store nodes of one list in a hash set, then find the first matching node in the other list.	$O(N+M)$	$O(N)$
Length difference	Find both lengths, advance the longer list by the difference, then move both pointers together.	$O(N+M)$	$O(1)$
Optimal - Pointer switching	When pointer A ends, move it to headB; when B ends, move it to headA. They meet at the intersection or null.	$O(N+M)$	$O(1)$

Revision key: Reason: Both pointers travel exactly $N+M$ distance.

2. Detect a Cycle

Approach	Core idea	Time	Space
Hashing	Store visited node addresses. Reaching an already visited node proves a cycle.	$O(N)$	$O(N)$
Optimal - Floyd	Move slow by one and fast by two. If they meet, a cycle exists. If fast reaches null, no cycle exists.	$O(N)$	$O(1)$

Revision key: Pattern: Floyd's tortoise and hare.

3. Reverse Nodes in K Groups

Approach	Core idea	Time	Space
Optimal	For each group, locate the kth node, save the next group, disconnect and reverse the current group, then reconnect it. Leave fewer than k remaining nodes unchanged.	$O(N)$	$O(1)$

Revision key: Pointers: groupStart, kthNode, nextGroup and previousTail.

4. Palindrome Linked List

Approach	Core idea	Time	Space
Brute force	Store values in an array or stack and compare from both ends.	$O(N)$	$O(N)$
Optimal	Find the first half end, reverse the second half, compare both halves, and optionally restore the list.	$O(N)$	$O(1)$

Revision key: Important: Direct comparison without reversing fails because the first pointer moves forward, not backward.

5. Starting Point of a Cycle

Approach	Core idea	Time	Space
Hashing	The first node visited twice is the cycle entry.	$O(N)$	$O(N)$
Optimal - Floyd	First make slow and fast meet. Reset slow to head, then move both one step at a time. Their next meeting is the cycle start.	$O(N)$	$O(1)$

Revision key: Key result: Distance from head to entry equals distance from meeting point to entry around the cycle.

6. Flatten a Linked List

Approach	Core idea	Time	Space
Brute force	Collect all values, sort them and create a new bottom-linked list.	$O(T \log T)$	$O(T)$
Optimal - Recursive merge	Flatten head.next recursively, then merge the current bottom list with the flattened list using the merge-two-sorted-lists technique.	Depends on lists	Recursion stack

Revision key: Key: Connect nodes through bottom pointers and set next pointers appropriately.